7주차

AI 에이전트와 MCP로 한계 넘기

서브에이전트 협업, AI 개발팀 구축, MCP 연동과 자동화

신구대학교 컴퓨터소프트웨어학과 · 앱프로그래밍응용 (2026-1)

오늘 배울 내용

이 수업의 3대 핵심 역량

역량	설명
바이브 코딩	AI 코딩 파트너를 활용, 자연어 프롬프트로 빠르게 개발
컴퓨팅 사고	문제 분해 → 패턴 인식 → 추상화 → 알고리즘적 해결
프로젝트 관리	요구사항 → 일정 계획 → 중간 보고 → 최종 발표

오늘의 학습 목표

1. **서브에이전트**를 생성하고 역할별로 코드 리뷰, 최적화, UX 개선을 수행할 수 있다.
2. **AI 개발팀**(PM, 백엔드, 프런트엔드, QA, AI 통합)을 서브에이전트로 구성하고 협업시킬 수 있다.
3. **MCP**의 개념을 이해하고, 노션·Context7·Playwright·깃허브 MCP를 설치하여 개발 환경을 자동화할 수 있다.

> 📖 *교재: 혼자 공부하는 바이브 코딩 with 클로드 코드 — Chapter 07 (p.258~301) & Chapter 08 (p.302~369)*

지난 주 복습

6주차에서 배운 것

- **API** — 프로그램과 AI 모델을 이어주는 다리, 간단한 요청으로 AI 기능 활용 (교재 p.225)
- **오픈라우터** — 여러 AI 모델을 한곳에서 비교·선택할 수 있는 API 통합 플랫폼 (교재 p.226)
- **API 키** — API에 접근하기 위한 비밀 열쇠, .env 파일에 보관하고 .gitignore로 보호 (교재 p.227)
- **.env / .gitignore** — API 키를 코드와 분리하여 안전하게 관리 (교재 p.233)
- **다중 AI 모델 연동** — 이미지 인식 + 텍스트 생성 AI를 결합한 '냉장고를 부탁해' 앱 (교재 p.243)
- **3단계 PRD** — 복잡한 앱을 단계별 계획(프로토타입 → 핵심기능 → 완성)으로 점진적 구현 (교재 p.247)

이번 주 핵심 질문

> 💬 *"API로 AI 모델의 힘을 빌렸는데, **여러 AI를 팀처럼 협업시키고, 외부 도구까지 자유롭게 연결**하려면 어떻게 해야 할까?"*

이번 주의 컴퓨팅 사고

컴퓨팅 사고의 **4가지 사고력**이 이번 주에 어떻게 적용되는지 살펴봅시다.

사고력	의미	이번 주 적용
분해	큰 문제를 작은 단위로 나누기	개발팀 역할을 PM·백엔드·프런트·QA·AI통합으로 분해
패턴 인식	반복되는 구조 찾기	서브에이전트 생성 패턴(이름→권한→모델→배경색) 반복
추상화	핵심만 뽑아 정리하기	MCP로 외부 서비스 연결을 "표준 규약"으로 추상화
알고리즘	순서대로 절차 밟기	MCP 설치→인증→연결 확인→활용의 단계적 절차

💡 이번 주의 핵심은 **"추상화"**! MCP라는 표준 프로토콜 하나로 노션, 깃허브, 브라우저 등 전혀 다른 서비스를 동일한 방식으로 연결하는 것이 추상화의 핵심입니다.

3대 역량, 이번 주 적용

역량	이번 주 활동
바이브 코딩	서브에이전트에게 자연어로 역할 부여, MCP 연동으로 개발 환경 자동화
컴퓨팅 사고	개발팀 역할 분해, MCP 연결 구조를 추상화하여 이해
프로젝트 관리	5W1H로 요구사항 정의, 서브에이전트 팀으로 역할 분담 및 협업

바이브 코딩 = 서브에이전트·MCP로 개발을 **자동화** (도구)
컴퓨팅 사고 = 역할 분해와 MCP 표준화를 **추상화**하여 이해 (사고)
프로젝트 관리 = AI 팀 구성과 5W1H로 **체계적 개발 계획** (계획)

에이전트란? (교재 p.258)

AI 에이전트의 개념

에이전트(Agent)는 '대리인'이라는 뜻으로, 주어진 **목표와 규칙**에 따라 **스스로 판단하고 작업을 수행**하는 존재입니다.

AI 서브에이전트

- 클로드 코드에서 **특정 작업을 전문적으로 수행**하는 독립형 AI 도우미
- 각각 고유한 **역할, 권한, 전문 분야**를 가짐
- 메인 AI(클로드 코드)가 서브에이전트에게 **작업을 위임**

서브에이전트 vs 커스텀 명령어

구분	커스텀 명령어	서브에이전트
성격	정해진 규칙에 따라 실행	상황을 판단하여 유연하게 대응
비유	레시피대로 따라하는 요리사	재료를 보고 창의적으로 요리하는 셰프
적합한 작업	반복적·정형화된 작업	복잡한 판단이 필요한 작업

서브에이전트 생성하기 (교재 p.260)

서브에이전트 생성 절차

```
/agents → Create new agent → Project(.claude/agents/)
→ Generate with Claude → 프롬프트 입력
→ 도구 권한 설정 → 모델 선택 → 배경색 설정
```

이번 주에 만드는 3가지 서브에이전트

에이전트	역할	권한	모델	배경색
code-bug-analyzer	코드 리뷰 & 버그 분석	Read-only	Opus	🔵 Blue
performance-optimizer	성능 최적화	All tools	Inherit	🟢 Green
ux-design-advisor	UX 디자인 개선	All tools	Inherit	🟠 Orange

💡 코드 리뷰어는 **Read-only** 권한으로 코드를 읽기만 하고 수정은 하지 않습니다. 분석 전문 에이전트는 읽기 전용이 안전합니다.

서브에이전트 협업 (교재 p.273)

3개 에이전트 동시 활용

- 코드 리뷰어가 **버그와 보안 이슈** 발견
- 최적화 에이전트가 **성능 개선** 포인트 제안
- UX 에이전트가 **사용성 개선** 사항 도출

직렬 처리 vs 병렬 처리 (교재 p.274)

처리 방식	설명	적합한 경우
직렬 처리	순서대로 하나씩 실행	앞 작업 결과가 뒤 작업에 필요할 때
병렬 처리	여러 작업을 동시에 실행	작업 간 의존성이 없을 때

> 클로드 코드가 **작업 성격에 따라 직렬/병렬 방식을 자동으로 선택**합니다. 코드 리뷰 3개를 동시에 요청하면 병렬로, 리뷰 후 수정은 직렬로 진행됩니다.

작업 결과 백업 (교재 p.280)

백업 명령어

```
현재 상태를 백업해 줘
```

- 클로드 코드가 **backup 디렉터리**를 자동 생성하고 현재 파일 상태를 복사
- 서브에이전트가 코드를 수정하기 전에 백업하면 안전

> 💡 서브에이전트가 코드를 대량으로 수정하기 전에 반드시 백업하세요! 백업 명령 한 줄이면 현재 상태를 안전하게 보존할 수 있습니다.

셀프 체크 — 07-1 (교재 p.281)

1. 커스텀 명령어와 에이전트의 기술적 차이점으로 옳은 것을 고르세요.

① 커스텀 명령어는 .md 파일로 저장되지만 에이전트는 .json 파일로 저장된다.

② 에이전트는 독립적인 AI 모델을 선택할 수 있지만 커스텀 명령어는 메인 세션의 모델을 공유한다.

③ 커스텀 명령어가 에이전트보다 더 복잡한 작업 플로우를 처리할 수 있다.

④ 에이전트는 적용을 위해 재시작이 필요하지만 커스텀 명령어는 바로 사용할 수 있다.

2. 에이전트 생성 시 '4. Inherit from parent' 옵션을 선택했을 때의 동작으로 옳은 것을 고르세요.

① 항상 가장 저렴한 모델을 자동 선택한다.

② 메인 세션에서 사용 중인 모델을 그대로 상속받는다.

③ 부모 에이전트가 사용하는 모델을 복사한다.

④ Claude의 최신 모델로 자동 업그레이드된다.

3. 에이전트에게 Read-only 권한만 부여했을 때 수행할 수 없는 작업을 모두 고르세요.

① 코드 파일 내용 읽기 및 분석 ② 버그 리포트 생성

③ 발견한 버그 수정 ④ 새로운 테스트 파일 생성

AI 개발팀 구축 (교재 p.284)

소프트웨어 개발팀 구성과 역할

실제 개발팀의 역할을 **5개 서브에이전트**로 재현합니다.

에이전트	역할	배경색
product-manager-prd	제품 기획, 요구사항 정의(PRD)	Red
backend-architect	백엔드 설계, 서버·API 구현	Blue
frontend-developer	프런트엔드 UI/UX 구현	Green
qa-engineer	테스트, 품질 보증	Yellow
ai-integration-specialist	AI 모델 연동 전문가	Purple

> 5개 에이전트 모두 **Sonnet 모델** + **All tools** 권한으로 생성합니다. 빠른 응답 속도가 필요한 개발 작업에는 Sonnet이 적합합니다. (교재 p.288)

5W1H 프레임워크 (교재 p.289)

제품 요구사항 정리

서브에이전트 팀에게 작업을 지시하기 전에 **5W1H 프레임워크**로 요구사항을 명확히 합니다.

항목	질문	예시 (AI 공감 다이어리)
Who	누가 구현하나?	백엔드·프런트엔드·QA 에이전트
What	무엇을 만드나?	일기 작성 → AI가 감정 분석 → 공감 답변
When	언제 만드나?	지금 바로 개발 시작, 즉시 완성
Where	어디서 사용하나?	웹 브라우저
Why	왜 필요한가?	혼자 감정 정리가 어려운 사람에게 위로
How	어떻게 구현하나?	OpenRouter API + 감정 분석 AI

💡 5W1H 프레임워크로 요구사항을 명확히 정의한 뒤 역할별 에이전트에게 작업을 지시하면, 실제 팀 개발과 동일한 협업이 가능합니다.

에이전트 팀 협업 사례 (교재 p.289)

사례 1: AI 공감 다이어리

1. **사용자** → 5W1H로 요구사항 정의
2. **백엔드** → OpenRouter API로 감정 분석 로직 구현
3. **프런트엔드** → 따뜻한 일기장 느낌의 UI 구현
4. **QA** → 기능 테스트 수행

사례 2: PDF 요약 AI (교재 p.294)

1. **PM** → 5W1H 기반 요구사항 정의
2. **백엔드** → PDF 텍스트 추출 로직
3. **AI 통합** → 요약 및 핵심 키워드 추출 AI 연동
4. **프런트엔드** → PDF 업로드 → 요약 결과 표시 UI
5. **QA** → 전체 기능 테스트

> 5개 서브에이전트를 **총동원**하면 역할별 전문성을 살려 빠르고 체계적인 개발이 가능합니다.

셀프 체크 — 07-2 (교재 p.300)

1. AI 공감 다이어리 개발에서 백엔드 개발과 프런트엔드 개발보다 품질 보증(QA) 테스트 시간이 더 오래 걸린 이유로 가장 적절한 것을 고르세요.

① QA 에이전트가 더 느린 AI 모델을 사용하기 때문이다.

② QA 에이전트는 이전 에이전트들의 작업 결과를 모두 검토해야 하기 때문이다.

③ QA 에이전트만 인터넷 연결이 필요하기 때문이다.

④ QA 에이전트는 파일 읽기 권한이 없기 때문이다.

2. PDF 요약 도구에서 제품 기획 관리자 에이전트가 생성한 PRD를 다른 에이전트들이 참조하는 방식으로 옳지 **않은** 것을 고르세요.

① PRD가 프로젝트 폴더에 생성되어 각 에이전트들이 접근한다.

② 에이전트들이 순차적, 혹은 병렬로 작업하며 PRD를 참조하게 된다.

③ PRD 파일 경로를 에이전트별로 지정하지 않아도 된다.

④ PRD는 '.claude/agents/' 폴더에 자동으로 복사되어 공유된다.

MCP란? (교재 p.304)

MCP(Model Context Protocol)

MCP는 AI 모델이 외부 데이터, 도구, 서비스와 연결할 때 사용하는 **표준화된 통신 규약**입니다.

API vs MCP (교재 p.305)

구분	API	MCP
비유	특정 가게 하나의 대화 창구	여러 가게를 한꺼번에 관리하는 표준 규격
연결 대상	특정 서비스 1개	다양한 데이터·도구를 통합 관리
사용자	개발자가 직접 코드 작성	AI가 스스로 도구를 선택하여 활용

클라이언트-서버 모델 (교재 p.306)

- **클로드 코드** = MCP **클라이언트** (요청하는 쪽)
- **노션, 깃허브, Playwright 등** = MCP **서버** (서비스를 제공하는 쪽)

> 💡 MCP 덕분에 클로드 코드가 노션에 문서를 쓰고, 깃허브에 코드를 올리고, 브라우저를 자동 제어할 수 있습니다!

로컬 MCP vs 원격 MCP (교재 p.315)

두 가지 MCP 유형

구분	로컬 MCP	원격 MCP
실행 위치	내 컴퓨터 안에서 직접 실행	인터넷상 외부 서버에서 실행
인터넷	필요 없음	반드시 필요
설정	조금 더 복잡	간단 (URL만 지정)
속도	빠름 (로컬 실행)	네트워크에 따라 다름
보안	개인정보 보호에 유리	인증 절차 필요

원격 MCP 명령어 구조

```
claude mcp add --transport http [이름] [서버 URL]
```

로컬 MCP 명령어 구조

```
claude mcp add [이름] [옵션] -- [MCP 실행 명령]
```

MCP 설치 명령어 분석 (교재 p.316)

Sequential Thinking MCP 설치 명령어

```
claude mcp add sequential-thinking -s local
  -- cmd /c npx -y
  @modelcontextprotocol/server-sequential-thinking@latest
```

부분	의미
`sequential-thinking`	클로드 코드에서 이 MCP를 구분하기 위한 이름
`-s local`	현재 프로젝트에만 적용 (scope 설정)
`--`	클로드 코드 옵션과 MCP 실행 명령의 구분자
`cmd /c`	윈도우 환경에서 명령 실행 (맥/리눅스에서는 생략)
`npx -y`	Node.js 패키지를 자동 설치하여 실행
`@model...@latest`	패키지명@버전 (최신 버전)

💡 윈도우 파워셸 환경에서는 `cmd /c` 부분을 삭제합니다!

인증이란? (교재 p.316)

왜 인증이 필요한가?

외부 서비스와 MCP를 연결할 때, 접속하려는 사람이 **정말 허가된 사용자인지** 확인하기 위해 **인증**(Authentication)이 필요합니다.

인증 방식

- 원격 MCP → 인터넷을 통해 통신하므로 **인증 절차** 필수
- 로컬 MCP → 내 컴퓨터에서 직접 실행하므로 **인증 불필요**

> **원격 MCP**는 인터넷 통신이므로 API 키, OAuth 등 인증이 필수이고, **로컬 MCP**는 내 컴퓨터에서 실행하므로 별도 인증 없이 바로 사용할 수 있습니다.

셀프 체크 — 08-1 (교재 p.321)

1. MCP와 API의 차이점으로 가장 적절한 설명을 고르세요.

① API는 특정 기능을 요청하는 방식이고, MCP는 AI가 외부 시스템과 직접 소통하게 하는 프로토콜

② MCP는 API보다 속도가 빠르다

③ API는 유료이고 MCP는 무료이다

④ MCP는 클로드 코드에서만 사용 가능하다

2. 로컬 MCP 서버 설치 명령어에서 `--` 구분자의 역할로 옳은 것을 고르세요.

① 주석을 표시하는 기호

② 클로드 코드 옵션과 MCP 서버 실행 명령을 구분

③ 버전 정보를 나타내는 표시

④ 에러 처리를 위한 구분자

3. Sequential Thinking MCP를 사용했을 때와 일반 추론의 차이점을 서술하세요.

MCP를 찾는 방법 (교재 p.324)

원하는 MCP 서버 검색

1. **구글 검색** — `"[서비스명] MCP"` 검색 → 깃허브 문서 확인
2. **깃허브 저장소** — README에서 설치 명령어 확인

이번 주에 설치하는 MCP 서버들

MCP 서버	역할	유형
Notion	노션 문서 읽기/쓰기	원격 MCP
Sequential Thinking	순차적 추론으로 체계적 분석	로컬 MCP
Context7	최신 문서를 참고하도록 지식 연결	원격 MCP
Playwright	웹 자동화 테스트 (브라우저 제어)	로컬 MCP
GitHub	깃허브 저장소 생성·업로드	원격 MCP

> MCP는 누구나 만들고 배포할 수 있어 현재 **수천 가지**가 존재합니다. 구글 검색과 깃허브가 가장 효과적인 탐색 방법입니다. (교재 p.314)

테스트와 버전 관리 (교재 p.323)

왜 테스트가 필요한가?

- 코드가 의도한 대로 실행되는지, 오류가 없는지 확인하는 과정
- **Playwright** 같은 자동화 도구를 사용하면 AI가 반복적인 테스트를 대신 수행

왜 버전 관리가 필요한가?

- 코드의 모든 변화를 기록하고, 이전 상태로 되돌릴 수 있게 함
- **깃허브**(GitHub)와 MCP를 연결하면 AI가 자동으로 코드를 업로드

깃허브와 Git 명령어 (교재 p.342)

단계	명령어	설명
① **추가**(add)	`git add`	파일을 준비 공간(staging area)에 추가
② **커밋**(commit)	`git commit`	현재 상태를 로컬 저장소에 저장
③ **푸시**(push)	`git push`	로컬 → 원격 저장소(깃허브)에 업로드

배포와 데이터베이스 (교재 p.346)

배포(Deploy)란?

내 애플리케이션을 외부 인터넷에 연결된 **서버에 올리는** 과정입니다.

- **Vercel** — 깃허브에 올리면 자동으로 빌드하고 배포해 주는 클라우드 플랫폼
- 배포하면 어디서든 **URL로 접속** 가능

로컬 스토리지 vs 클라우드 스토리지

구분	로컬 스토리지	클라우드 데이터베이스
저장 위치	브라우저 내부	인터넷 서버
공유	불가능 (같은 브라우저만)	가능 (어디서나)
데이터 유지	캐시 삭제 시 사라짐	영구 보관

Supabase

- 클라우드 **데이터베이스**를 간단히 자동화해 주는 플랫폼
- 무료 플랜 제공, API로 데이터 저장/조회 가능

셀프 체크 — 08-2 (교재 p.345)

1. Playwright MCP가 테스트 완료 후 생성하는 폴더 이름을 고르세요.

① .test-results ② .playwright-mcp ③ .automation ④ .screenshots

2. 깃허브에 코드를 업로드하는 순서로 올바른 것을 고르세요.

① commit → add → push ② push → add → commit

③ add → commit → push ④ add → push → commit

3. MCP 서버 주소를 찾는 가장 효과적인 방법 두 가지를 설명하세요.

셀프 체크 — 08-3 (교재 p.368)

1. 로컬 스토리지와 클라우드 데이터베이스의 가장 큰 차이점은?

① 로컬 스토리지가 더 빠르다.

② 데이터베이스는 여러 기기에서 데이터를 공유할 수 있다.

③ 로컬 스토리지가 더 안전하다.

④ 데이터베이스는 무료가 없다.

2. Vercel 배포 시 404 에러가 발생한 이유와 해결 방법으로 옳은 것을 고르세요.

① 서버 오류 – 재시작 필요

② index.html 파일 부재 – 파일명 변경 필요

③ 인증 실패 – 재로그인 필요

④ 네트워크 오류 – 연결 확인 필요

3. Supabase의 테이블과 엑셀 시트의 유사점과 차이점을 설명하세요.

실습 시작 안내

실습 흐름

```
실습 ① 서브에이전트 3종 생성 (코드 리뷰 · 최적화 · UX)
  ↓
실습 ② 서브에이전트 협업 테스트 & 백업
  ↓
실습 ③ 노션 MCP 연결 & 리서치 노트 생성
  ↓
실습 ④ Sequential Thinking MCP 설치 & 성능 비교
  ↓
실습 ⑤ Context7 · Playwright MCP 설치 & 자동 테스트
```

준비물

- 6주차에서 만든 **'냉장고를 부탁해' 프로젝트** (Study-06 폴더)
- **노션 계정** (무료 가입)
- **Node.js** 설치 완료

실습 ① 서브에이전트 3종 생성 (교재 p.260)

코드 리뷰어 에이전트 생성

```
/agents → Create new agent → Project
→ Generate with Claude
```

프롬프트 입력 (교재 p.262)

```
코드 리뷰어: 코드를 읽고 버그는 없는지, 코딩 규칙에 따라
올바르게 작성되었는지를 점검하고 성능 최적화를 제안하는
전문 코드 품질 검토자
```

설정

항목	code-bug-analyzer
도구 권한	Read-only tools
모델	Opus
배경색	🔵 Blue

실습 ① (계속) — 최적화 & UX 에이전트

performance-optimizer (교재 p.268)

```
최적화 전문가: 애플리케이션의 작동을 원활하게 개선하고,
속도를 빠르게 만들며, 병목 지점을 찾아서 해결하는
시스템 성능 최적화 엔지니어
```

- 권한: **All tools** / 모델: **Inherit from parent** / 배경색: 🟢 Green

ux-design-advisor (교재 p.271)

```
UX 디자이너: 사용자가 쉽고 편하게 이용할 수 있도록
화면 디자인, 버튼 배치, 에러 메시지를 개선하는
사용자 경험 전문가
```

- 권한: **All tools** / 모델: **Inherit from parent** / 배경색: 🟠 Orange

[확인] 체크리스트

- [ ] `/agents` 명령어에서 3개 에이전트가 모두 표시되는가?
- [ ] `.claude/agents/` 폴더에 3개의 `.md` 파일이 생성되었는가?

실습 ② 서브에이전트 협업 테스트 (교재 p.273)

3개 에이전트로 '냉장고를 부탁해' 앱 분석

```
code-bug-analyzer로 '냉장고를 부탁해' 애플리케이션 코드 전체를
리뷰한 뒤, 발견한 문제를 performance-optimizer가
수정해서 성능 최적화한 다음, ux-design-advisor가
사용자 경험을 개선하게 해 줘.
```

기대 결과

- 코드 리뷰어 → **버그와 개선점** 발견
- 최적화 에이전트 → 발견된 문제를 **성능 최적화**
- UX 에이전트 → **사용자 경험 개선** 사항 도출

실습 ② (계속) — 백업 & 확인

백업 후 개선 적용 (교재 p.280)

```
현재 상태를 백업해 줘.
```

- 클로드 코드가 **backup 폴더**를 생성하고 현재 상태를 자동 복사합니다.

[확인] 체크리스트

- [ ] 3개 에이전트의 분석 리포트를 확인했는가?
- [ ] backup 폴더가 생성되었는가?

실습 ③ 노션 MCP 연결 (교재 p.306)

노션 MCP 설치

```
claude mcp add --transport http notion https://mcp.notion.com/mcp
```

연결 확인

1. 설치 후 브라우저에서 노션 로그인 및 **인증**(Authorize) 완료
2. 클로드 코드에서 `/mcp` 입력 → **notion: connected** 확인
3. **View tools** → 15개 도구 확인 (notion-search, notion-fetch 등)

노션 MCP로 리서치 노트 만들기 (교재 p.313)

```
클로드 코드의 업데이트, 변경 사항을 검색하고 요약해 줘.
그 결과를 노션 MCP를 이용해 노션에 저장해 줘.
```

[확인] 체크리스트

- [ ] `/mcp`에서 notion이 connected 상태인가?
- [ ] 노션에 문서가 생성되었는가?

실습 ④ Sequential Thinking MCP (교재 p.316)

설치 명령어 (윈도우 cmd 환경)

```
claude mcp add sequential-thinking -s local
  -- cmd /c npx -y
  @modelcontextprotocol/server-sequential-thinking@latest
```

> ⚠️ *윈도우 파워셸 환경에서는* `cmd /c` *부분을 삭제하세요!*

설치 확인

```
claude mcp list
```

실습 ④ (계속) — 순차적 추론 성능 비교 (교재 p.318)

순차적 추론 vs 일반 추론

```
내 웹 포트폴리오 방문자의 체류 시간을 지금의 2배로 늘리고 싶어.
두 가지 문서를 작성해.
1. 노션 MCP를 이용해 '체류 시간 늘리기'로 저장해 줘.
2. Sequential Thinking MCP를 이용해서 체계적으로 구상하고
   '체류 시간 늘리기-체계적 구상'으로 저장해 줘.
```

- 일반 추론: 총 85줄 / 순차적 추론: 총 **222줄** (약 2.6배 상세)

> 💡 Sequential Thinking은 복잡한 문제를 여러 단계로 나눠 사고하므로 일반 추론보다 더 체계적이고 상세한 결과를 생성합니다.

실습 ⑤ Context7 MCP (교재 p.326)

Context7 MCP 설치

```
claude mcp add --transport http context7
  https://mcp.context7.com/mcp
  --header "CONTEXT7_API_KEY: <발급받은 키>"
```

- Context7 홈페이지: https://www.context7.com
- 가입 → API 키 발급 → 설치

실습 ⑤ (계속) — Playwright MCP (교재 p.328)

Playwright MCP 설치

```
claude mcp add playwright -s local
  -- cmd /c npx @playwright/mcp@latest
```

> ⚠️ *윈도우 파워셸 환경에서는* `cmd /c` *부분을 삭제하세요!*

설치 확인

```
claude mcp list
```

- notion, sequential-thinking, context7, playwright 모두 **Connected** 확인

실습 ⑤ (계속) — 쇼핑 리스트 앱 & 자동 테스트

쇼핑 리스트 앱 만들기 (교재 p.329)

```
쇼핑 리스트 앱을 만들어 줘.
아이템 추가, 삭제, 체크 기능이 있는 간단한 웹 UI로
로컬 브라우저에서 실행되게 해 줘.
```

Playwright MCP로 자동 테스트 (교재 p.330)

```
Playwright MCP를 사용해서 이 쇼핑 리스트 앱의
모든 기능을 자동으로 테스트해 줘.
아이템 추가, 삭제, 체크 기능이 제대로 작동하는지 확인해 줘.
```

[확인] 체크리스트

- [ ] `shopping-list.html` 파일이 생성되고 브라우저에서 작동하는가?
- [ ] Playwright가 자동으로 아이템 추가/삭제/체크를 테스트했는가?
- [ ] `.playwright-mcp` 폴더에 스크린샷이 저장되었는가?

실습 중 자주 발생하는 문제

문제	원인	해결 방법
MCP 설치 후 connected 안 됨	Node.js 미설치 또는 버전 오래됨	`node -v` 확인, v18 이상 필요
노션 인증 실패	브라우저 팝업 차단	팝업 허용 후 재시도
`cmd /c` 에러 (파워셸)	파워셸에서 `cmd /c` 불필요	`cmd /c` 부분 제거
Sequential Thinking 응답 없음	npx 패키지 다운로드 지연	네트워크 확인 후 재시도
Playwright 브라우저 안 열림	Chromium 미설치	`npx playwright install chromium` 실행
깃허브 MCP 인증 실패	API 토큰 만료 또는 권한 부족	토큰 재발급 (repo, user, project 권한)

핵심 키워드 — 이론 07-1 (교재 p.281)

> 📖 ***07-1**: 6가지 키워드*

- **에이전트** — 사람이 직접 지시하지 않아도 목표와 규칙에 따라 스스로 판단하고 작업을 수행하는 AI 대리 프로그램
- **AI 서브에이전트** — 클로드 코드에서 특정 작업을 전문적으로 수행하도록 만든 독립형 AI 도우미
- **/agents** — 클로드 코드의 전용 명령어, 새로운 서브에이전트를 생성하거나 관리
- **코드 리뷰** — 코드의 버그와 규칙 위반을 점검하고 개선점을 제안하는 품질 검토 과정
- **최적화** — 프로그램의 속도, 자원 사용, 효율성을 향상시키는 성능 개선 작업
- **UX 디자인** — 사용자가 쉽고 편리하게 이용할 수 있도록 화면 구성과 인터페이스를 설계하는 과정

핵심 키워드 — 이론 07-2 (교재 p.300)

> 📖 ***07-2**: 5가지 키워드*

- **제품 기획 관리자** — 제품의 목표, 기능, 사용자 요구사항을 정의하고 전체 개발 일정을 관리
- **백엔드 개발자** — 서버, 데이터베이스 설계와 API 연동 등 핵심 로직을 구현하는 기술 담당자
- **프런트엔드 개발자** — 사용자가 보는 화면과 인터페이스를 설계하고 구현하는 UI 개발자
- **품질 보증 엔지니어** — 기능과 성능을 테스트해 버그를 수정하고 서비스의 안정성을 확보
- **AI 통합 전문가** — AI 모델을 연동, 최적화하기 위해 기술 통합을 담당

핵심 키워드 — 실습 ① (교재 Chapter 08)

> 📖 ***08-1**: 6가지 키워드 (교재 p.321)*

- **MCP** — AI 모델이 외부 데이터, 도구, 서비스와 연결할 때 사용하는 표준화된 통신 규약
- **클라이언트-서버 모델** — 클로드 코드(클라이언트)가 MCP 서버에 요청하고 응답받는 구조
- **로컬 MCP** — MCP 중개 프로그램을 내 컴퓨터 안에서 직접 실행하는 방식
- **원격 MCP** — 그 중개 프로그램이 인터넷상 외부 서버에서 실행하는 방식
- **인증** — 접속하려는 사람이 정말 허가된 사용자인지 확인하는 절차
- **이중 하이픈** `--` — 두 글자 이상인 옵션 이름을 표시하거나, 명령어와 옵션을 구분할 때 쓰이는 기호

핵심 키워드 — 실습 08-2 (교재 p.345)

> 📖 *08-2: 7가지 키워드*

- **테스트** — 코드가 의도한 대로 실행되는지, 오류가 없는지 확인하는 과정
- **버전 관리** — 코드의 변경 이력을 기록하고, 필요할 때 이전 상태로 되돌릴 수 있게 함
- **Context7** — 클로드 코드가 항상 최신 문서를 참고하도록 돕는 지식 연결 도구
- **Playwright** — 웹 사이트를 자동으로 테스트하고 UI를 검증하는 브라우저 제어 도구
- **추가(add)** — 깃허브에 업로드할 파일들을 선택해서 준비 공간에 추가
- **커밋(commit)** — 준비된 파일을 로컬 저장소에 저장
- **푸시(push)** — 로컬 저장소에 커밋된 파일을 원격 저장소로 업로드

핵심 키워드 — 실습 08-3 (교재 p.368)

> 📖 ***08-3**: 6가지 키워드*

- **배포** — 내 애플리케이션을 외부 인터넷에 연결된 서버에 올리는 과정
- **Vercel** — 깃허브에 코드를 올리면 자동으로 빌드·배포해 주는 클라우드 플랫폼
- **로컬 스토리지** — 다른 컴퓨터나 브라우저에는 공유되지 않고 캐시를 지우면 사라지는 저장소
- **클라우드 스토리지** — 어디서나 동일한 데이터를 불러올 수 있는 중앙 서버 역할
- **Supabase** — 데이터베이스 구축을 자동화해 주는 클라우드 데이터베이스 플랫폼
- **테이블** — 데이터를 표 형태로 저장한 구조 (데이터베이스 안의 개별 엑셀 시트와 유사)

오늘의 마무리 체크리스트

수업이 끝나기 전에 아래 항목을 확인해 보세요!

- [ ] **서브에이전트** 3종(코드 리뷰어·최적화·UX)을 생성하고 역할을 설명할 수 있다
- [ ] **서브에이전트 협업**으로 '냉장고를 부탁해' 앱을 분석하고 개선 리포트를 확인했다
- [ ] **MCP의 개념**을 이해하고 API와의 차이를 설명할 수 있다
- [ ] **노션 MCP**를 연결하고 리서치 노트를 자동 생성했다
- [ ] **Sequential Thinking MCP**를 설치하고 일반 추론과 성능을 비교했다
- [ ] **Playwright MCP**로 쇼핑 리스트 앱을 자동 테스트했다
- [ ] "내가 오늘 배운 것" 2줄 메모

> ✅ 서브에이전트 분석 리포트 캡처 + MCP 연결 목록(claude mcp list) 캡처를 교수님께 제출하면 오늘 수업 완료!

최신 업데이트 — AI 에이전트 & MCP (2026년 4월 기준)

> 📢 *교재 출간 이후 주요 업데이트를 정리합니다!*

Claude Code 에이전트 기능

구분	교재 (2025)	최신 (2026.04)
서브에이전트	`/agents` 메뉴로 생성	**에이전트 마켓** 추가 (커뮤니티 공유)
모델 선택	Opus / Sonnet / Inherit	**Claude 4 계열** (Opus 4, Sonnet 4) 추가
MCP 생태계	수백 개 MCP 서버	**수천 개** 이상으로 확대
Playwright	@playwright/mcp@latest	**v0.0.70** 릴리스, CLI+Skills 모드 병행 지원

MCP 프로토콜 변동

- **MCP 거버넌스** — Anthropic 단독 → **Linux Foundation** 프로젝트로 이관 (오픈 거버넌스)
- **SSE 전송 방식 → Streamable HTTP로 전환 완료** (stdio 병행 지원)
- **Context7** — API 키 기반 인증 지원 (무료 플랜 유지)
- **GitHub MCP** — PAT(Personal Access Token) 외 OAuth 지원 추가

과제 안내

이번 주 과제

서브에이전트 3종을 생성하고, MCP 2개 이상을 설치하여 연동 결과를 제출하세요.

과제 요구사항

- **서브에이전트 3종** 생성 화면 캡처 (`/agents` 목록)
- 서브에이전트 **협업 분석 리포트** 일부 캡처
- **MCP 연결 목록** 캡처 (`claude mcp list`)
- MCP를 활용한 **결과물** 1개 이상 (노션 문서, Playwright 테스트 스크린샷 등)

다음 주 예고

> ***8주차**: 중간 평가 — 바이브 코딩 활용 능력 객관식 평가*

이번 주에 배운 **서브에이전트**와 **MCP**는 AI 개발 환경의 확장 핵심입니다. 서브에이전트로 팀을 구성하고, MCP로 외부 도구를 연결하면 하나의 AI가 아닌 **"AI 개발팀 + 도구 생태계"**를 구축할 수 있습니다!